



AuraStore: The Modern Consumer App

AuraStore is a blazing-fast, visually stunning e-commerce storefront built with a modern decoupled architecture. It is designed to prove that consumer-facing applications can deliver both uncompromising performance and beautiful, fluid user experiences.

By leveraging Next.js Partial Prerendering (PPR), a headless CMS, and optimistic UI updates, AuraStore provides an app-like feel on the web.



Core Concept & Vision

- **Performance First:** Instant initial loads utilizing Next.js Turbopack and Partial Prerendering (PPR).
- **Decoupled Architecture:** Frontend presentation is entirely separated from the backend inventory and payment logic.
- **Fluid UX:** Delightful micro-interactions, seamless page transitions, and optimistic state updates ensure the app never feels sluggish.
- **Real Payments:** End-to-end checkout flow with secure webhook validation.



Tech Stack

Frontend (UI & Experience)

- **Framework:** Next.js (App Router, Turbopack, PPR enabled)
- **Styling:** Tailwind CSS
- **UI Components:** shadcn/ui (built on Radix UI for accessibility)
- **Animations:** Framer Motion (page transitions and layout animations)
- **Data Fetching & Caching:** TanStack Query (React Query)
- **Notifications:** Sonner (toast notifications for cart and checkout events)
- **Icons:** Lucide React

Backend & Infrastructure

- **Headless CMS:** Strapi (manages product catalog, categories, and order history)
- **Payment Gateway:** Razorpay (handles transactions and server-to-server webhook verification)
- **Database:** PostgreSQL / SQLite (managed via Strapi)



System Architecture

AuraStore operates on a headless e-commerce model:

1. **Content Delivery:** The Next.js frontend fetches static product data from Strapi. Using PPR, a static shell is served instantly to the user, while dynamic components (like the cart) stream in.
2. **Client-Side State:** TanStack Query intercepts user actions (like adding to cart) to update the UI optimistically before the server responds.
3. **Checkout Flow:**
 - The user initiates a checkout from the cart.

- Next.js securely requests an `order_id` from Razorpay.
- The Razorpay client-side modal captures the payment.
- Razorpay triggers a backend webhook to Next.js.
- Next.js verifies the webhook signature and securely updates the order status in Strapi.



Getting Started

Follow these instructions to set up the project locally.

Prerequisites

- Node.js (v18.17 or higher)
- npm, pnpm, or yarn
- A running Strapi instance (local or cloud)
- Razorpay account (Test mode API keys)



1. Clone the Repository

```
git clone https://github.com/yourusername/aurastore.git
cd aurastore
```

2. Environment Variables

Create a `.env.local` file in the root of your Next.js project and populate it with the following keys:

```
# App URL
NEXT_PUBLIC_APP_URL=http://localhost:3000

# Strapi CMS Connection
NEXT_PUBLIC_STRAPI_API_URL=http://localhost:1337
STRAPI_API_TOKEN=your_strapi_api_read_write_token

# Razorpay Integration
NEXT_PUBLIC_RAZORPAY_KEY_ID=your_razorpay_test_key_id
RAZORPAY_KEY_SECRET=your_razorpay_test_key_secret
RAZORPAY_WEBHOOK_SECRET=your_secure_webhook_secret
```

3. Install Dependencies

```
npm install
# or
pnpm install
```

4. Run the Development Server

AuraStore uses Turbopack for lightning-fast local development.

```
npm run dev --turbo
```

Navigate to `http://localhost:3000` to view the storefront.

Key Directory Structure

```
aurastore/
├── src/
│   ├── app/                # Next.js App Router (Pages, Layouts, API routes)
│   │   ├── api/            # Serverless functions (Razorpay webhooks, etc.)
│   │   └── (storefront)/   # Public-facing store routes
│   ├── components/        # Reusable UI components
│   │   ├── ui/            # shadcn/ui base components
│   │   └── shared/        # Custom complex components (ProductCard, CartDrawer)
│   ├── lib/               # Utility functions, API clients, TanStack hooks
│   └── styles/            # Global CSS and Tailwind configuration
├── public/               # Static assets
└── tailwind.config.ts    # Tailwind theme and design tokens
```

Security Notes

- **Never** expose your `RAZORPAY_KEY_SECRET` or `STRAPI_API_TOKEN` to the client.
- All payment success states must be verified via the Razorpay webhook endpoint (`/api/webhooks/razorpay`) using crypto signature verification before marking an order as complete in Strapi.